

3-Е ИЗДАНИЕ

ИЗУЧАЕМ PYTHON

ПРОГРАММИРОВАНИЕ ИГР,
ВИЗУАЛИЗАЦИЯ ДАННЫХ, ВЕБ-ПРИЛОЖЕНИЯ

ЭРИК МЭТИЗ

ПРОДАНО
СВЫШЕ 500 000
ЭКЗЕМПЛЯРОВ!



by Eric Matthes

PYTHON CRASH COURSE

2ND EDITION

**A Hands-On,
Project-Based Introduction
to Programming**



**no starch
press**

San Francisco

ЭРИК МЭТИЗ

ИЗУЧАЕМ PYTHON

ПРОГРАММИРОВАНИЕ ИГР,
ВИЗУАЛИЗАЦИЯ ДАННЫХ,
ВЕБ-ПРИЛОЖЕНИЯ

3-Е ИЗДАНИЕ



Санкт-Петербург • Москва • Екатеринбург • Воронеж
Нижний Новгород • Ростов-на-Дону
Самара • Минск

2021

ББК 32.973.2-018.1
УДК 004.43
М97

Мэттиз Эрик

М97 Изучаем Python: программирование игр, визуализация данных, веб-приложения. 3-е изд. — СПб.: Питер, 2021. — 512 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-1528-0

«Изучаем Python» — это самое популярное в мире руководство по языку Python. Вы сможете максимально быстро освоить Python, научитесь писать программы, устранять ошибки и создавать работающие приложения.

В первой части книги вы познакомитесь с основными концепциями программирования, такими как переменные, списки, классы и циклы, а простые упражнения познакомят вас с шаблонами чистого кода. Вы узнаете, как делать программы интерактивными и как протестировать код, прежде чем добавлять в проект. Во второй части вы примените новые знания на практике и создадите три проекта: аркадную игру в стиле Space Invaders, визуализацию данных с удобными библиотеками Python и простое веб-приложение, которое можно быстро развернуть онлайн.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018.1
УДК 004.43

Права на издание получены по соглашению с No Starch Press. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1593279288 англ.

ISBN 978-5-4461-1528-0

© 2019 by Eric Matthes.

Title of English-language original: Python Crash Course, 2nd Edition:
A Hands-On, Project-Based Introduction to Programming,
ISBN 978-1-59327-928-8, published by No Starch Press.

© Перевод на русский язык ООО Издательство «Питер», 2021

© Издание на русском языке, оформление ООО Издательство
«Питер», 2021

© Серия «Библиотека программиста», 2021

Оглавление

Отзывы о книге	10
Об авторе	11
О научном редакторе	11
Предисловие к третьему изданию	13
Благодарности	16
Введение	17
Для кого написана эта книга?	17
Чему эта книга вас научит?	18
Онлайн-ресурсы	19
Почему именно Python?	19
От издательства	20
ЧАСТЬ I. ОСНОВЫ.	21
Глава 1. Первые шаги	22
Подготовка среды программирования	22
Python в разных операционных системах	23
Запуск программы Hello World	28
Решение проблем с установкой	30
Запуск программ Python в терминале	31
Итоги	32
Глава 2. Переменные и простые типы данных	33
Что происходит при запуске hello_world.py	33
Переменные	33
Строки	37
Числа	43
Комментарии	46

Дзен Python	47
Итоги	49
Глава 3. Списки	50
Что такое список?	50
Индексы начинаются с 0, а не с 1	51
Изменение, добавление и удаление элементов	53
Упорядочение списка	59
Ошибки индексирования при работе со списками	62
Итоги	63
Глава 4. Работа со списками	64
Перебор всего списка	64
Предотвращение ошибок с отступами	67
Создание числовых списков	71
Работа с частью списка	75
Кортежи	80
Стиль программирования	82
Итоги	85
Глава 5. Команды if	86
Проверка условий	87
Команды if	93
Использование команд if со списками	100
Оформление команд if	104
Итоги	104
Глава 6. Словари	106
Простой словарь	106
Работа со словарями	107
Перебор словаря	114
Вложение	120
Итоги	127
Глава 7. Ввод данных и циклы while	128
Как работает функция input()	128
Циклы while	132
Использование цикла while со списками и словарями	138
Итоги	142
Глава 8. Функции	143
Определение функции	143
Передача аргументов	145

Возвращаемое значение	151
Передача списка	156
Хранение функций в модулях	163
Стилевое оформление функций	167
Итоги	168
Глава 9. Классы	170
Создание и использование класса	171
Работа с классами и экземплярами	175
Наследование	180
Импортирование классов	187
Стандартная библиотека Python	193
Оформление классов	194
Итоги	195
Глава 10. Файлы и исключения	196
Чтение из файла	196
Запись в файл	204
Исключения	206
Сохранение данных	215
Итоги	221
Глава 11. Тестирование	222
Тестирование функции	222
Тестирование класса	229
Итоги	235
ЧАСТЬ II. ПРОЕКТЫ	237
Программирование игры на языке Python	238
Визуализация данных	238
Веб-приложения	238
ПРОЕКТ 1. ИГРА «ИНОПЛАНЕТНОЕ ВТОРЖЕНИЕ»	239
Глава 12. Инопланетное вторжение	240
Планирование проекта	241
Установка Pygame	241
Создание проекта игры	241
Добавление изображения корабля	245
Рефакторинг: методы <code>_check_events()</code> и <code>_update_screen()</code>	249
Управление кораблем	251
В двух словах	259

Стрельба	260
Итоги	267
Глава 13. Осторожно, пришельцы!	268
Анализ проекта	268
Создание пришельца	269
Построение флота	272
Перемещение флота	277
Уничтожение пришельцев	281
Завершение игры	285
Определение исполняемых частей игры	291
Итоги	292
Глава 14. Ведение счета	293
Добавление кнопки Play	293
Повышение сложности	299
Подсчет очков	302
Итоги	317
ПРОЕКТ 2. ВИЗУАЛИЗАЦИЯ ДАННЫХ	318
Глава 15. Генерирование данных	319
Установка matplotlib	320
Построение простого графика	320
Случайное блуждание	330
Моделирование бросков кубиков в Plotly	338
Итоги	346
Глава 16. Загрузка данных	347
Формат CSV	347
Построение карт с глобальными наборами данных: формат JSON	361
Итоги	373
Глава 17. Работа с API	374
Использование API веб-приложений	374
Итоги	391
ПРОЕКТ 3. ВЕБ-ПРИЛОЖЕНИЯ	392
Глава 18. Знакомство с Django	393
Подготовка к созданию проекта	393
Начало работы над приложением	398
Построение других страниц	413
Итоги	422

Глава 19. Учетные записи пользователей	423
Редактирование данных	423
Создание учетных записей пользователей	435
Редактирование данных	444
Итоги	452
Глава 20. Оформление и развертывание приложения	454
Оформление приложения Learning Log	454
Развертывание Learning Log	465
Итоги	481
Послесловие	483
ПРИЛОЖЕНИЯ	485
Приложение А. Установка Python и диагностика проблем	486
Python в Windows	486
Python в macOS	488
Python в системе Linux	489
Ключевые слова Python и встроенные функции	489
Приложение Б. Текстовые редакторы	491
Настройка Sublime Text	492
Другие текстовые редакторы и IDE	493
Приложение В. Помощь и поддержка	496
Первые шаги	496
Поиск в интернете	497
IRC (Internet Relay Chat)	499
Slack	500
Discord	501
Приложение Г. Git и контроль версий	502
Установка Git	502
Создание проекта	503
Игнорирование файлов	503
Инициализация репозитория	504
Проверка статуса	504
Добавление файлов в репозиторий	505
Закрепление	505
Просмотр журнала	506
Второе закрепление	506
Отмена изменений	507
Извлечение предыдущих закреплений	508
Удаление репозитория	510

ОТЗЫВЫ О КНИГЕ

Интересно наблюдать за тем, как No Starch Press создает будущую классику, которая по праву может занять место рядом с более традиционными книгами по программированию. Эта книга — одна из них.

Грег Лейден (Greg Laden), ScienceBlogs

Автор рассматривает довольно сложные проекты и излагает материал в последовательной, логичной и приятной манере, которая привлекает читателя к теме.

Full Circle Magazine

Хорошая подача материала с доступными объяснениями фрагментов кода. Книга ведет читателя вперед шаг за шагом, строя все более сложный код и объясняя все происходящее на этом пути.

FlickThrough Reviews

Изучение Python с этой книгой оставило в высшей степени положительные впечатления! Отличный вариант, если вы только начинаете изучать Python.

Mikke Goes Coding

Об авторе

Эрик Мэтиз, преподаватель физики и математики, живет на Аляске и ведет курс Python начального уровня. Эрик пишет программы с пяти лет, а в настоящее время занимается разработкой продуктов, которые исправляют недочеты в системе образования и помогают использовать возможности программных продуктов с открытым кодом в системе образования. В свободное время занимается альпинизмом и проводит время с семьей.

О научном редакторе

Кеннет Лав — преподаватель и программист Python с многолетним стажем. Он выступал с докладами и лекциями на конференциях, занимался профессиональной подготовкой, работал внештатным программистом Python и Django, а в настоящее время работает программистом в O'Reilly Media. Кеннет является одним из создателей пакета `django-braces`, предоставляющего удобные примеси (mixins) для представлений на базе классов Django. Желающие могут читать его сообщения в Твиттере (@kennethlove).

*Моему отцу, который никогда не жалел
времени, чтобы ответить на мои вопросы
по программированию, и Эверу, который
только начинает задавать мне свои вопросы*

Предисловие к третьему изданию

Предыдущее издание «Изучаем Python: программирование игр, визуализация данных, веб-приложения» получило множество положительных отзывов. Тираж книги составил более 500 тысяч экземпляров, включая переводы на 8 языков. Я получал письма и сообщения как от читателей, которым было всего десять лет, так и от пенсионеров, желающих освоить программирование для собственного удовольствия. Материал книги используется для обучения в школах и колледжах. Студенты, которым приходится иметь дело с более сложными учебниками, используют мою книгу как дополнительный источник для своих занятий; по их мнению, книга является хорошим подспорьем в учебе. Разные люди используют ее для повышения своей квалификации и для работы над параллельными проектами. Короче говоря, книга используется для самых разнообразных целей — как я и надеялся.

Я был очень рад тому, что мне представилась возможность поработать над следующим изданием. Python — достаточно зрелый язык, но он продолжает развиваться, как и все языки. При переработке материала книги я старался сделать его более доступным и компактным. Необходимости в изучении Python 2 больше нет, поэтому новое издание полностью сосредоточено на Python 3. Установка многих пакетов Python упростилась, поэтому упростились и инструкции по установке и настройке. Я добавил несколько тем, которые, на мой взгляд, будут полезны читателям, обновил некоторые разделы и представил новые, более простые средства для решения некоторых задач в языке Python. Также были доработаны разделы, в которых некоторые аспекты языка были представлены недостаточно точно. Все проекты были полностью обновлены; в них используются только популярные библиотеки с качественным сопровождением, которыми вы можете пользоваться при построении собственных проектов.

Ниже приведена краткая сводка изменений в третьем издании:

- ❑ В главе 1 инструкции по установке Python были упрощены для пользователей всех основных операционных систем. Теперь я рекомендую использовать текстовый редактор Sublime Text, который популярен как среди новичков, так и среди профессионалов и работает во всех операционных системах.
- ❑ В главе 2 приведено более подробное описание того, как переменные реализованы в языке Python. Переменные описываются как текстовые *метки* для значений, что позволяет читателю лучше понять поведение пере-

менных в Python. В книге широко используются f-строки, появившиеся в Python 3.6, — намного более простой и удобный механизм использования значений в строках. В Python 3.6 также появилась возможность использования символов подчеркивания для представления больших чисел (например, `1_000_000`); она также включена в это издание. Множественное присваивание переменных было представлено в одном из проектов первого издания; описание было обобщено и перемещено в главу 2 для удобства читателей. Наконец, в эту главу также включена понятная система соглашений для представления констант в Python.

- ❑ В главе 6 представлен метод `get()` для чтения значений из словаря, который может вернуть значение по умолчанию в случае отсутствия ключа.
- ❑ Проект «Инопланетное вторжение» (главы 12–14) теперь полностью базируется на использовании классов. Сама игра представляет собой класс (вместо серии функций, как в предыдущем издании). Это сильно упрощает общую структуру игры, уменьшает число вызовов функций и необходимых параметров. Читатели, знакомые с первым изданием, оценят простоту нового подхода, основанного на классах. Ругаме теперь может устанавливаться однострочной командой во всех системах, а читатель может выбрать между запуском игры в полноэкранном или оконном режиме.
- ❑ В проектах визуализации данных инструкции по установке Matplotlib упростились для всех операционных систем. Визуализации, базирующиеся на Matplotlib, используют функцию `subplots()`, более удобную для обучения построению сложных визуализаций. В проекте Rolling Dice из главы 15 используется Plotly — библиотека визуализации с простым синтаксисом, качественным сопровождением и красивым выводом результатов с широкими возможностями настройки.
- ❑ В главе 16 метеорологический проект переведен на данные NOAA, которые продемонстрировали большую стабильность за прошедшие годы, чем сайт, использованный в первом издании. Картографический проект теперь посвящен глобальной сейсмической активности; к моменту завершения проекта будет построена великолепная визуализация, наглядно изображающая границы тектонических плит по координатам всех землетрясений за заданный период времени. Вы научитесь наносить на карту любые наборы данных, содержащие географические координаты.
- ❑ В главе 17 для визуализации активности, связанной с Python, в проектах с открытым кодом на GitHub используется библиотека Plotly.
- ❑ Проект Learning Log (главы 18–20) построен на базе новейшей версии Django, а для его оформления используется новейшая версия Bootstrap. Процесс развертывания проекта в Heroku был упрощен за счет использования пакета `django-heroku`, и в нем используются переменные среды вместо редактирования файлов `settings.py`. Такое решение получается более простым и лучше соответ-

ствуется тому, как профессиональные программисты развертывают современные проекты Django.

- Приложение А было полностью переработано. В него были включены рекомендации по установке Python. В приложение Б включены подробные рекомендации по настройке Sublime Text и приведены краткие описания основных текстовых редакторов и интегрированных сред, используемых в наши дни. В приложении В приведены ссылки на более новые и популярные сетевые ресурсы для получения помощи, а приложение Г предлагает небольшой учебный курс по применению Git для контроля версий.

Спасибо за то, что читаете эту книгу! Если у вас появятся вопросы или вы захотите поделиться своим мнением, не стесняйтесь и пишите.

Благодарности

Эта книга никогда бы не появилась на свет без великолепных, чрезвычайно профессиональных сотрудников издательства *No Starch Press*. Билл Поллок (Bill Pollock) предложил мне написать вводный учебник, и я глубоко благодарен ему за это предложение. Тайлер Ортман (Tyler Ortman) помог привести в порядок мои идеи на ранней стадии подготовки чернового варианта. Лиз Чедвик (Liz Chadwick) и Лесли Шен (Leslie Shen) предоставили бесценные отзывы на исходные варианты каждой главы, а Энн Мэри Уокер (Anne Marie Walker) помогла прояснить многие части книги. Райли Хоффман (Riley Hoffman) отвечал на все вопросы, которые возникали у меня в процессе создания книги, и терпеливо превращал мою работу в прекрасный завершённый продукт.

Также хочу поблагодарить Кеннета Лава, научного редактора книги. Я познакомился с Кеннетом на конференции PyCon, и его энтузиазм в отношении языка и сообщества Python с тех пор неизменно оставался для меня источником профессионального вдохновения. Кеннет вышел за рамки простой проверки фактов; он следил за тем, чтобы книга помогала начинающим программистам сформировать основательное понимание языка Python и программирования в целом. Вместе с тем ответственность за все оставшиеся неточности лежит исключительно на мне.

Я хочу поблагодарить своего отца, который познакомил меня с программированием в раннем возрасте и не побоялся, что я сломаю его оборудование. Также хочу сказать спасибо своей жене Эрин за поддержку и помощь во время работы над книгой и своему сыну Эверу, чья любознательность постоянно служит мне примером.

Введение

У каждого программиста найдется своя история о том, как он написал свою первую программу. Я начал изучать программирование еще в детстве, когда мой отец работал на Digital Equipment Corporation, одну из ведущих компаний современной эры вычислительной техники. Я написал свою первую программу на компьютере, который был собран моим отцом из набора компонентов в подвале дома. Компьютер представлял собой системную плату (без корпуса), подключенную к клавиатуре, а в качестве монитора использовалась простейшая электронно-лучевая трубка. Моей первой программой стала игра по отгадыванию чисел, которая выглядела примерно так:

```
Я загадал число! Попробуйте отгадать мое число: 25
Слишком мало! Следующая попытка: 50
Слишком много! Следующая попытка: 42
Верно! Хотите сыграть снова? (да/нет) нет
Спасибо за игру!
```

Никогда не забуду, как это было здорово — моя семья играла в написанную мной игру, и все работало точно так, как я задумал.

Мои ранние переживания имели далекоидущие последствия. Очень приятно построить нечто, предназначенное для конкретной цели; нечто, успешно решающее свою задачу. Программы, которые я пишу сейчас, намного серьезнее моих детских попыток, но чувство удовлетворения, которое я ощущаю от вида работающей программы, остается практически тем же.

Для кого написана эта книга?

Цель этой книги — как можно быстрее ввести читателя в курс дела, чтобы он начал писать на Python работоспособные программы (игры, визуализации данных и веб-приложения), и одновременно заложить основу в области программирования, которая пригодится ему на протяжении всей жизни. Книга написана для людей любого возраста, которые прежде никогда не программировали на Python или вообще никогда не программировали. Если вы хотите быстро изучить азы программирования, чтобы сосредоточиться на интересных проектах, а также проверить свое понимание новых концепций на содержательных задачах, — эта книга для вас. Книга также

прекрасно подходит для преподавателей, желающих предложить вводный курс программирования, основанный на проектах. Наконец, если вы студент, которому хочется иметь более доступное введение в Python, чем предложенный ему учебник, эта книга также упростит вашу учебу.

Чему эта книга вас научит?

Цель книги — сделать вас хорошим программистом вообще и хорошим программистом Python в частности. Процесс обучения будет эффективным, и вы приобретете много полезных привычек, так как я представляю основательное введение в общие концепции программирования. После того как вы перевернете последнюю страницу, вы будете готовы к знакомству с более серьезными возможностями Python, а изучение вашего следующего языка программирования тоже упростится.

В части I книги будут представлены базовые концепции программирования, которые необходимо знать для написания программ Python. Эти концепции ничем не отличаются от тех, которые рассматриваются в начале изучения почти любого языка программирования. Вы познакомитесь с разными видами данных и возможностями хранения данных в списках и словарях. Вы научитесь создавать коллекции данных и эффективно работать с этими коллекциями. В частности, циклы `while` и `if` позволяют выполнять определенные фрагменты кода, если некоторое условие истинно, и выполнять другие фрагменты в противном случае — эти конструкции очень сильно помогают при автоматизации процессов.

Вы научитесь получать входные данные от пользователя, чтобы ваши программы стали интерактивными, и выполнять их до тех пор, пока пользователь остается активным. Также вы узнаете, как написать функции для многократного выполнения некоторых частей ваших программ, чтобы вы один раз запрограммировали некоторое действие, а потом могли использовать его столько раз, сколько потребуется. Затем эта концепция будет распространена на более сложное поведение с классами, что позволит даже относительно простым программам реагировать на множество разнообразных ситуаций. Вы научитесь писать программы, корректно обрабатывающие многие типичные ошибки. После знакомства с базовыми концепциями мы напишем несколько коротких программ для решения конкретных задач. Наконец, вы сделаете первые шаги на пути к программированию среднего уровня: вы научитесь писать тесты для своего кода, чтобы продолжать разработку программ, не беспокоясь о возможном внесении ошибок. Вся информация части I подготовит вас к более сложным и масштабным проектам.

В части II знания, полученные в части I, будут применены для построения трех проектов. Вы можете взяться за любые из этих проектов в том порядке, который лучше подходит для вас. В первом проекте (главы 12–14) будет создана игра-стрелялка в стиле классического хита *Space Invaders*, состоящая из многих уровней с нарастающей сложностью. После завершения этого проекта вы будете знать многое из того, что необходимо знать для разработки собственных 2D-игр.

Второй проект (главы 15–17) познакомит вас с визуализацией данных. Чтобы разобраться в огромных объемах доступной информации, специалисты по анализу данных применяют различные средства визуализации. Вы будете работать с наборами данных, генерируемыми в программах; наборами данных, загруженными из сетевых источников; и наборами данных, которые загружаются вашей программой автоматически. После завершения этого проекта вы сможете писать программы, обрабатывающие большие наборы данных и строящие визуальные представления сохраненной информации.

В третьем проекте (главы 18–20) будет построено небольшое веб-приложение Learning Log. Этот проект позволяет вести журнал новых идей и концепций, которые вы узнали в ходе изучения конкретной темы. Пользователь приложения сможет вести разные журналы по разным темам, создавать учетные записи и начинать новые журналы. Вы также узнаете, как развернуть свой проект в интернете, чтобы любой желающий мог работать с ним откуда угодно.

Онлайн-ресурсы

Все информационные ресурсы, прилагаемые к книге, доступны в интернете по адресу <https://nostarch.com/pythoncrashcourse2e/> или http://ehmatthes.github.io/pcc_2e/:

- ❑ **Инструкции по настройке.** Они совпадают с инструкциями, приведенными в книге, но в них включены активные ссылки, относящиеся к различным фазам настройки. Если у вас возникнут какие-либо проблемы с подготовкой и настройкой, обращайтесь к этому ресурсу.
- ❑ **Обновления.** Python, как и все языки, постоянно развивается. Я отслеживаю наборы обновлений, так что если что-то не работает — обращайтесь к этому ресурсу и проверьте, не изменилось ли что-то.
- ❑ **Решения к упражнениям.** Не жалейте времени на самостоятельное решение задач из разделов «Упражнения». Но если вы оказались в тупике и не знаете, что делать, ответы к большинству упражнений доступны в интернете.
- ❑ **Памятки.** Полный набор памяток, содержащих краткие сводки по основным концепциям.

Почему именно Python?

Каждый год я задумываюсь над тем, продолжать ли мне работать на Python или же перейти на другой язык — вероятно, более новый в мире программирования. И все же я продолжаю работать на Python по многим причинам. Язык Python невероятно эффективен: программы, написанные на нем, делают больше, чем многие на других языках и в меньшем объеме кода. Синтаксис Python также позволяет писать «чистый» код. Ваш код будет легко читаться, у вас будет меньше проблем с отладкой и расширением программ по сравнению с другими языками.

Python используется для разных целей: для создания игр, построения веб-приложений, решения бизнес-задач и разработки внутренних инструментов для всевозможных интересных проектов. Python также широко применяется в научной области для теоретических исследований и решения прикладных задач.

Впрочем, одной из самых важных причин для использования Python для меня остается сообщество Python, состоящее из невероятно разных и благожелательных людей. Сообщество играет исключительно важную роль в программировании, потому что программирование не является сугубо индивидуальным делом. Многим из нас, даже самым опытным программистам, приходится обращаться за советом к коллегам, которые уже решали похожие задачи. Существование связанного, доброжелательного сообщества помогает решать задачи, и сообщество Python готово прийти на помощь людям, для которых Python является первым языком программирования.

Python — замечательный язык, так давайте браться за дело!

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Часть I

ОСНОВЫ

В части I этой книги представлены базовые концепции, необходимые для написания программ на языке Python. Многие из этих концепций встречаются во всех языках программирования, поэтому они пригодятся вам на протяжении всей карьеры в программировании.

В главе 1 вы установите Python на свой компьютер и запустите свою первую программу, которая выводит на экран сообщение `Hello world!`.

В главе 2 вы научитесь хранить информацию в переменных, работать с текстовыми и числовыми данными.

В главах 3 и 4 вы познакомитесь со списками. Списки позволяют хранить любой объем информации в одной переменной, что повышает эффективность работы с данными. Вы сможете работать с сотнями, тысячами и даже миллионами значений всего в нескольких строках кода.

В главе 5 будут представлены команды `if`. С их помощью вы сможете написать код, который делает что-то одно, если некоторое условие истинно, и что-то другое, если условие не выполняется.

Глава 6 показывает, как использовать словари Python, связывающие разные виды информации. Словари, как и списки, могут содержать столько информации, сколько вы захотите в них поместить.

В главе 7 вы научитесь получать данные от пользователей, чтобы ваши программы стали интерактивными. Также в этой главе описаны циклы `while`, многократно выполняющие блоки кода, пока некоторое условие остается истинным.

В главе 8 вы займетесь написанием функций — именованных блоков кода, которые решают конкретную задачу и запускаются тогда, когда потребуется.

В главе 9 представлены классы, предназначенные для моделирования реальных объектов: собак, кошек, людей, машин, ракет и т. д. С их помощью вы сможете представить в своем коде любые сущности, реальные или абстрактные.

Глава 10 научит вас работать с файлами и обрабатывать ошибки, чтобы ваши программы не завершались аварийно. Вы узнаете, как сохранить данные перед закрытием программы и снова загрузить их при запуске программы. В этой главе рассматриваются исключения Python; с их помощью вы сможете предвидеть возможные ошибки и организовать их корректную обработку в программах.

В главе 11 вы научитесь писать тесты для своего кода. Тесты проверяют, что ваша программа работает так, как было задумано. В результате вы сможете дорабатывать свои программы, не беспокоясь о возможном внесении новых ошибок. Тестирование — один из первых навыков, отличающих новичка от программиста среднего уровня.

1

Первые шаги

В этой главе вы запустите свою первую программу на языке Python, `hello_world.py`. Сначала вы проверите, установлен ли Python на вашем компьютере, и если нет — установите его. Также будет установлен текстовый редактор для подготовки программ Python. Текстовые редакторы распознают код Python и выделяют синтаксические конструкции во время работы, упрощая понимание структуры кода разработчиком.

Подготовка среды программирования

Поддержка Python слегка отличается в разных операционных системах, поэтому вы должны учитывать некоторые аспекты. В этой главе мы проверим, что Python правильно установлен в вашей системе.

Версии Python

Каждый язык программирования развивается с появлением новых идей и технологий, и разработчики Python неустанно трудятся над тем, чтобы сделать язык более мощным и гибким. На момент написания книги новейшей была версия 3.7, но все программы в книге должны нормально работать в версии 3.6. В этом разделе вы определите, установлена ли поддержка Python в вашей системе и нужно ли установить новую версию. В приложении А содержится подробное руководство по установке новейших версий Python во всех основных операционных системах.

Некоторые старые проекты Python продолжают использовать Python 2, но вам стоит остановиться на Python 3. Если в вашей системе установлена только версия Python 2, скорее всего, она нужна для поддержки других программ, необходимых для вашей системы. Мы оставим эту установку без изменений и позаботимся о том, чтобы в системе была более свежая версия для работы.

Выполнение фрагментов кода Python

Интерпретатор Python может выполняться в терминальном окне и позволяет опробовать фрагменты кода Python без сохранения и запуска всей программы.

В этой книге встречаются фрагменты следующего вида:

```
❶ >>> print("Hello Python interpreter!")  
Hello Python interpreter!
```

Приглашение >>> означает, что используется окно терминала, а жирным шрифтом выделена команда, которую вы вводите и выполняете нажатием клавиши Enter. Большинство примеров в книге представляет небольшие самостоятельные программы, которые запускаются из редактора, а не в терминале, потому что именно так вы будете писать большую часть своего кода. Но в некоторых случаях базовые концепции будут продемонстрированы серией фрагментов в терминальном сеансе Python, чтобы более эффективно показать отдельные концепции. Каждый раз, когда в листинге встречаются три угловые скобки ❶, это означает, что перед вами вывод терминального сеанса. Вскоре мы опробуем возможность программирования в интерпретаторе для вашей системы.

Также текстовый редактор будет использоваться для создания простой программы *Hello World!*. В мире программирования издавна принято начинать освоение нового языка с программы, выводящей на экран сообщение *Hello world!* — считается, что это принесет удачу. Даже такая простая программа выполняет вполне конкретную функцию. Если она запускается в вашей системе, то и любая программа, которую вы напишете на Python, тоже должна запускаться нормально.

О текстовом редакторе Sublime Text

Sublime Text — простой текстовый редактор, который устанавливается во всех основных операционных системах. Sublime Text позволяет запускать практически любые программы прямо из редактора (вместо терминала), а код выполняется в терминальном сеансе, встроенном в окно Sublime Text, что упрощает просмотр вывода.

Редактор Sublime Text особенно удобен для начинающих, но многие профессиональные программисты также пользуются им. Если вы привыкнете к нему во время изучения Python, возможно, вы будете пользоваться им и при переходе на более крупные и сложные проекты. Политика лицензирования Sublime Text более чем либеральна: вы можете бесплатно пользоваться редактором сколько угодно долго, но автор требует приобрести лицензию, если программа вам понравилась и вы собираетесь использовать ее в будущем.

В приложении Б приведена информация о других вариантах; возможно, вам стоит бегло просмотреть его. Если вы хотите быстро перейти к программированию, используйте Sublime Text на первых порах и рассмотрите переход на другие редакторы, когда у вас появится некоторый опыт программирования. В этой главе я опишу процесс установки Sublime Text для вашей операционной системы.

Python в разных операционных системах

Python является кросс-платформенным языком программирования; это означает, что он работает во всех основных операционных системах. Любая программа на

языке Python, написанная вами, должна выполняться на любом современном компьютере с установленной поддержкой Python. Впрочем, способы настройки Python для разных операционных систем слегка отличаются.

В этом разделе вы узнаете, как подготовить Python к работе в вашей системе. Сначала вы проверите, установлена ли новая версия Python в вашей системе, и если нет — установите ее. Затем вы установите Sublime Text. Процедура состоит всего из двух шагов, отличающихся для разных операционных систем.

Затем вы установите программу *Hello World* и устраните любые неполадки. Этот процесс будет описан для всех операционных систем, так что в итоге в вашем распоряжении появится простая и удобная среда программирования на Python.

Python в системе Windows

Windows далеко не всегда включает поддержку Python. Скорее всего, вам придется загрузить и установить Python, а затем загрузить и установить текстовый редактор Sublime Text.

Установка Python

Для начала проверьте, установлена ли поддержка Python в вашей системе. Откройте окно командной строки: введите `command` в меню Пуск или щелкните на рабочем столе с нажатой клавишей Shift и выберите команду `Open command window here`. Введите в окне командной строки команду `python` в нижнем регистре. Если на экране появится приглашение `>>>`, значит, в системе установлена поддержка Python. Впрочем, скорее всего, вместо приглашения появится сообщение об ошибке, в котором говорится, что команда `python` не опознана системой.

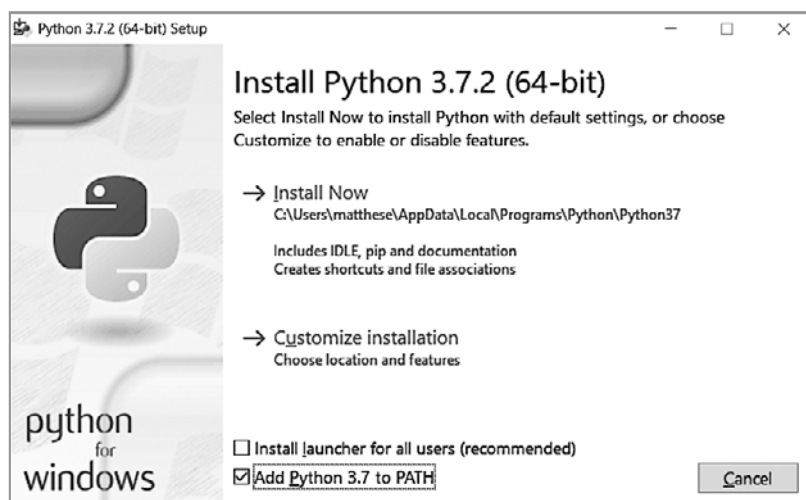


Рис. 1.1. Не забудьте установить флажок Add Python to PATH

В таком случае (а также если будет выведена версия Python менее 3.6) загрузите программу установки Python для Windows. Откройте страницу <https://python.org/> и наведите указатель мыши на ссылку **Downloads**. Появляется кнопка для загрузки новейшей версии Python. Щелкните на кнопке, которая запускает автоматическую загрузку правильного установочного пакета для вашей системы. После того как загрузка файла будет завершена, запустите программу установки. Не забудьте установить флажок **Add Python to PATH**; это упростит правильную настройку системы. На рис. 1.1 изображено окно мастера установки с установленным флажком.

Запуск Python в терминальном сеансе

Откройте окно командной строки и введите команду `python` в нижнем регистре. Если на экране появится приглашение Python (`>>>`), значит, система Windows обнаружила установленную версию Python:

```
C:\> python
Python 3.7.2 (v3.7.2:9a3ffc0492, Dec 23 2018, 23:09:28) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

ПРИМЕЧАНИЕ Если вы не увидите этот (или похожий) вывод, обращайтесь к более подробным инструкциям по установке в приложении А.

Введите в сеансе Python следующую строку и убедитесь в том, что на экране появился вывод `Hello Python interpreter!`

```
>>> print("Hello Python interpreter!")
Hello Python interpreter!
>>>
```

Каждый раз, когда вы захотите выполнить фрагмент кода Python, откройте окно командной строки и запустите терминальный сеанс Python. Чтобы закрыть терминальный сеанс, нажмите **Ctrl+Z**, а затем **Enter** или введите команду `exit()`.

Установка Sublime Text

Программу установки Sublime Text можно загрузить по адресу <https://sublimetext.com/>. Щелкните на ссылке загрузки и найдите программу установки для Windows. После того как программа установки будет загружена, запустите ее и подтвердите все настройки по умолчанию.

Python в системе macOS

В большинстве систем macOS поддержка Python уже установлена, но скорее всего, это будет какая-нибудь устаревшая версия, которую не стоит использовать для обучения. В этом разделе мы установим новейшую версию Python, а затем текстовый редактор Sublime Text и убедимся в том, что он правильно настроен.

Проверка наличия Python 3 в системе

Откройте терминальное окно (команда **Applications ▸ Utilities ▸ Terminal**). Также можно нажать **Command+пробел**, ввести **terminal** и нажать **Enter**. Чтобы проверить, установлена ли поддержка Python в вашей системе, введите команду **python** (со строчной буквы **p**). На экране появится информация о том, какая версия Python у вас установлена, и приглашение **>>>**, в котором можно вводить команды Python:

```
$ python
Python 2.7.15 (default, Aug 17 2018, 22:39:05)
[GCC 4.2.1 Compatible Apple LLVM 9.1.0 (clang-902.0.39.2)] on darwin
Type "help", "copyright", "credits", or "license" for more information.
>>>
```

Этот вывод сообщает, что Python 2.7.15 в настоящее время является версией Python по умолчанию, установленной на данном компьютере. Нажмите **Ctrl+D** или введите **exit()**, чтобы выйти из приглашения Python и вернуться к приглашению терминала.

Чтобы проверить наличие Python 3, попробуйте ввести команду **python3**. На экране может появиться сообщение об ошибке, но если из вывода следует, что в системе установлена версия Python 3.6 или более поздняя версия, вы можете перейти к разделу «Запуск Python в терминальном сеансе» этой главы. Если Python 3 не устанавливается по умолчанию, ее необходимо установить вручную. Каждый раз, когда в книге встречается команда **python**, вводите вместо нее команду **python3**, чтобы при запуске использовалась версия Python 3, а не Python 2; различия между этими версиями достаточно серьезны, чтобы при выполнении кода в книге с Python 2 могли возникнуть проблемы.

Если команда выводит любой номер версии, предшествующий 3.6, выполните инструкции в следующем разделе для установки новейшей версии.

Установка новейшей версии Python

Программа установки Python доступна на сайте <https://python.org/>. Наведите указатель мыши на ссылку **Downloads**. Появляется кнопка для загрузки новейшей версии Python. Щелкните на кнопке, которая запускает автоматическую загрузку правильного установочного пакета для вашей системы. После того как загрузка файла будет завершена, запустите программу установки.

Когда установка будет завершена, введите следующую команду в приглашении терминала:

```
$ python3 --version
Python 3.7.2
```

Если появится похожий результат, значит, все готово к работе с Python. Каждый раз, когда в книге встречается команда **python**, вводите вместо нее команду **python3**.

Запуск Python в терминальном сеансе

Для выполнения фрагментов кода Python можно открыть терминальное окно и ввести команду `python3`. Введите в терминальном сеансе следующую строку:

```
>>> print("Hello Python interpreter!")
Hello Python interpreter!
>>>
```

Сообщение выводится прямо в текущем терминальном окне. Вспомните, что интерпретатор Python закрывается комбинацией клавиш `Ctrl+D` или командой `exit()`.

Установка Sublime Text

Чтобы установить редактор Sublime Text, необходимо загрузить программу установки по адресу <https://sublimetext.com/>. Щелкните на ссылке **Download** и найдите программу установки для macOS. После того как программа установки будет загружена, откройте ее и перетащите значок Sublime Text в папку **Applications**.

Python в системе Linux

Системы семейства Linux ориентированы на программистов, поэтому поддержка Python уже установлена на большинстве компьютеров Linux. Люди, которые занимаются разработкой и сопровождением Linux, ожидают, что в какой-то момент вы займетесь программированием, и всячески способствуют этому. По этой причине для перехода к программированию вам почти ничего не придется устанавливать, а количество необходимых настроек будет минимальным.

Проверка версии Python

Откройте терминальное окно, запустив приложение **Terminal** в вашей системе (в Ubuntu нажмите клавиши `Ctrl+Alt+T`). Чтобы проверить, какая версия Python установлена в вашей системе, введите команду `python3` (со строчной буквы `p`). Если Python присутствует в системе, эта команда запустит интерпретатор Python. На экране появится информация о том, какая версия Python у вас установлена, и приглашение `>>>`, в котором можно вводить команды Python:

```
$ python3
Python 3.7.2 (default, Dec 27 2018, 04:01:51)
[GCC 7.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Этот вывод сообщает, что Python 3.7.2 в настоящее время является версией Python по умолчанию, установленной на данном компьютере. Нажмите `Ctrl+D` или введите `exit()`, чтобы выйти из приглашения Python и вернуться к приглашению терми-

нала. Каждый раз, когда в книге встречается команда `python`, вводите вместо нее команду `python3`.

Для запуска кода из книги необходима версия Python 3.6. Если в системе установлена более ранняя версия, обращайтесь к подробным инструкциям по установке в приложении А.

Запуск Python в терминальном сеансе

Для выполнения фрагментов кода Python можно открыть терминальное окно и ввести команду `python3`, как мы поступили при проверке версии. Сделайте то же самое, но на этот раз введите в терминальном сеансе следующую строку:

```
>>> print("Hello Python interpreter!")
Hello Python interpreter!
>>>
```

Сообщение выводится прямо в текущем терминальном окне. Вспомните, что интерпретатор Python закрывается комбинацией клавиш `Ctrl+D` или командой `exit()`.

Установка Sublime Text

В системе Linux редактор Sublime Text устанавливается из Ubuntu Software Center. Щелкните на значке **Ubuntu Software** в меню и найдите вариант **Sublime Text**. Щелкните на нем, чтобы установить, а потом запустить программу.

Запуск программы Hello World

После того как в вашей системе будут установлены последние версии Python и Sublime Text, все почти готово к запуску вашей первой программы Python, написанной в текстовом редакторе. Но перед этим необходимо убедиться в том, что в Sublime Text настроена правильная версия Python для вашей системы. После этого вы сможете написать программу *Hello World!* и запустить ее.

Настройка Sublime Text для использования правильной версии Python

Если команда `python` в вашей системе запускает Python 3, ничего настраивать не нужно и вы можете просто перейти к следующему разделу. Если же вы используете команду `python3`, необходимо настроить Sublime Text для использования правильной версии Python при запуске программ.

Щелкните на значке **Sublime Text** или проведите поиск **Sublime Text** в строке поиска вашей системы и запустите редактор. Выполните команду **Tools ▸ Build System ▸ New Build System**, которая откроет новый конфигурационный файл. Удалите текущее содержимое и введите следующий текст:

Python3.sublime-build

```
{
    "cmd": ["python3", "-u", "$file"],
}
```

Он приказывает Sublime Text использовать команду `python3` для запуска программных файлов Python. Сохраните этот файл под именем `Python3.sublime-build` в каталоге по умолчанию, который открывает Sublime Text при выполнении команды `Save`.

Запуск `hello_world.py`

Прежде чем писать первую программу, создайте где-нибудь в своей системе папку с именем `python_work` для своих проектов. В именах файлов и папок лучше использовать буквы нижнего регистра и символы подчеркивания в соответствии со схемой выбора имен в Python.

Откройте Sublime Text и сохраните пустой файл Python (**File ▶ Save As**) с именем `hello_world.py` в папке `python_work`. Расширение `.py` сообщает Sublime Text, что код в файле написан на языке Python; эта информация помогает редактору запустить программу и правильно выделить цветом элементы синтаксиса.

После того как файл будет сохранен, введите следующую строку в текстовом редакторе:

hello_world.py

```
print("Hello Python world!")
```

Если команда `python` работает в вашей системе, программу можно запустить командой меню **Tools ▶ Build** или комбинацией клавиш **Ctrl+B** (**Command+B** в macOS). Если вы настроили Sublime Text на использование другой команды вместо `python`, выберите команду меню **Tools ▶ Build System**, а затем **Python 3**. В дальнейшем вы сможете запускать свои программы командой **Tools ▶ Build** или комбинацией клавиш **Ctrl+B** (или **Command+B**).

В нижней части окна Sublime Text должно отображаться терминальное окно со следующим текстом:

```
Hello Python world!
[Finished in 0.1s]
```

Если вы не увидели это сообщение, проверьте каждый символ во введенной программе. Может, вы случайно набрали `print` с прописной буквы? Пропустили одну либо обе кавычки или круглые скобки? В языках программирования используется предельно конкретный синтаксис, и при малейшем его нарушении произойдет ошибка. Если программа так и не заработала, обращайтесь к следующему разделу.

2

Переменные и простые типы данных

В этой главе представлены разные виды данных, с которыми вы будете работать в своих программах Python. Вы также научитесь использовать переменные для представления данных в своих программах.

Что происходит при запуске `hello_world.py`

Давайте повнимательнее разберемся с тем, что же делает Python при запуске `hello_world.py`. Оказывается, даже для такой простой программы Python выполняет достаточно серьезную работу:

`hello_world.py`

```
print("Hello Python world!")
```

При выполнении этого кода выводится следующий текст:

```
Hello Python world!
```

Суффикс `.py` в имени файла `hello_world.py` указывает, что файл является программой Python. Редактор запускает файл в *интерпретаторе* Python, который читает программу и определяет, что означает каждое слово в программе. Например, когда интерпретатор обнаруживает слово `print`, он выводит на экран текст, заключенный в скобки.

Во время написания программы редактор выделяет цветом разные части программы. Например, он понимает, что `print` является именем функции, и выводит это слово одним цветом. С другой стороны, `"Hello Python world!"` не является кодом Python, поэтому этот текст выделяется другим цветом. Этот механизм, называемый *цветовым выделением синтаксиса*, сильно поможет вам, когда вы возьметесь за самостоятельное программирование.

Переменные

Попробуем использовать переменную в программе `hello_world.py`. Добавьте новую строку в начало файла и измените вторую строку:

hello_world.py

```
message = "Hello Python world!"  
print(message)
```

Запустите программу и посмотрите, что получится. Программа выводит уже знакомый результат:

```
Hello Python world!
```

В программу добавилась *переменная* с именем `message`. В каждой переменной хранится *значение*, то есть данные, связанные с переменной. В нашем случае значением является текст "Hello Python world!".

Добавление переменной немного усложняет задачу интерпретатора Python. Во время обработки первой строки он связывает текст "Hello Python world!" с переменной `message`. А когда интерпретатор доберется до второй строки, он выводит на экран значение, связанное с именем `message`.

Давайте немного расширим эту программу `hello_world.py`, чтобы она выводила второе сообщение. Добавьте в `hello_world.py` пустую строку, а после нее еще две строки кода:

```
message = "Hello Python world!"  
print(message)  
  
message = "Hello Python Crash Course world!"  
print(message)
```

Теперь при выполнении `hello_world.py` на экране должны появляться две строки:

```
Hello Python world!  
Hello Python Crash Course world!
```

Вы можете в любой момент изменить значение переменной в своей программе; Python постоянно отслеживает его текущее состояние.

Выбор имен и использование переменных

При работе с переменными в языке Python необходимо соблюдать некоторые правила и рекомендации. Нарушение правил приведет к ошибке; рекомендации всего лишь помогают писать более понятный и удобочитаемый код. Работая с переменными, помните о следующем:

- ❑ Имена переменных могут состоять только из букв, цифр и символов подчеркивания. Они могут начинаться с буквы или символа подчеркивания, но не с цифры. Например, переменной можно присвоить имя `message_1`, но не `1_message`.
- ❑ Пробелы в именах переменных запрещены, а для разделения слов в именах переменных используются символы подчеркивания. Например, имя `greeting_message` допустимо, а имя `greeting message` вызовет ошибку.

- ❑ Не используйте имена функций и ключевые слова Python в качестве имен переменных; иначе говоря, не используйте слова, которые зарезервированы в Python для конкретной цели, — например, слово `print` (см. раздел «Ключевые слова и встроенные функции Python»).
- ❑ Имена переменных должны быть короткими, но содержательными. Например, имя `name` лучше `n`, имя `student_name` лучше `s_n`, а имя `name_length` лучше `length_of_persons_name`.
- ❑ Будьте внимательны при использовании строчной буквы `l` и прописной буквы `O`, потому что они похожи на цифры `1` и `0`.

Вероятно, вы не сразу научитесь создавать хорошие имена переменных, особенно когда программы станут сложнее и интереснее. Но когда вы начнете писать свои программы и читать код, написанный другими разработчиками, ваши имена переменных станут более содержательными.

ПРИМЕЧАНИЕ Пока ограничьтесь именами переменных, записанными в нижнем регистре. Использование символов верхнего регистра не приведет к ошибке, но такие символы имеют специальное значение в именах переменных, и пока все же лучше обойтись без них.

Предотвращение ошибок в именах при использовании переменных

Каждый программист совершает ошибки, а большинство программистов совершает ошибки ежедневно. И хотя даже опытный программист не застрахован от ошибок, он знает, как эффективно реагировать на них. Рассмотрим типичную ошибку, которую вы довольно часто будете совершать на первых порах, и выясним, как эту ошибку исправить.

Для начала напишем код с намеренно внесенной ошибкой. Введите следующий фрагмент (неправильно написанное слово *mesage* выделено жирным шрифтом):

```
message = "Hello Python Crash Course reader!"
print(mesage)
```

Когда в программе происходит ошибка, интерпретатор Python всеми силами старается помочь вам в поиске причины. Если программа не выполняется нормально, интерпретатор предоставляет данные *трассировки* — информацию о том, в каком месте кода находился интерпретатор при возникновении проблем. Ниже приведен пример трассировки, которую выдает Python после случайной опечатки в имени переменной:

```
Traceback (most recent call last):
❶ File "hello_world.py", line 2, in <module>
❷     print(mesage)
❸ NameError: name 'mesage' is not defined
```

Строка ❶ сообщает, что ошибка произошла в строке 2 файла `hello_world.py`. Интерпретатор выводит номер строки, чтобы вам было проще найти ошибку ❷, и сообщает тип обнаруженной ошибки ❸. В данном случае была обнаружена ошибка в имени: переменная с указанным именем (`mesage`) не определена. Другими словами, Python не распознает имя переменной. Обычно такие ошибки возникают в том случае, если вы забыли присвоить значение переменной перед ее использованием или ошиблись при вводе имени.

Конечно, в данном примере в имени переменной во второй строке пропущена буква `s`. Интерпретатор Python не проверяет код на наличие опечаток, но следит за тем, чтобы имена переменных записывались одинаково. Например, вот что происходит, если имя `message` будет неправильно записано еще в одном месте кода:

```
message = "Hello Python Crash Course reader!"  
print(mesage)
```

На этот раз программа выполняется успешно!

```
Hello Python Crash Course reader!
```

Языки программирования не отличаются гибкостью, но орфография их совершенно не волнует. Как следствие, вам не нужно следить за тем, чтобы в именах переменных идеально соблюдались правила орфографии английского языка.

Многие ошибки программирования сводятся к простым опечаткам — случайной замене одного символа в одной строке программы. Если вы потратили много времени на поиск одной из таких ошибок, знайте, что вы не одиноки. Многие опытные и талантливые программисты тратят долгие часы на поиск подобных мелких ошибок. Нечто подобное будет часто происходить в ходе вашей работы — просто посмейтесь и идите дальше.

Переменные как метки

Переменные часто описывают как «ящики» для хранения значений. Такое сравнение может быть полезным на первых порах работы с переменными, но оно неточно описывает внутреннее представление переменных в Python. Намного правильнее представлять переменные как метки, которые можно назначать переменным. Также можно сказать, что переменная *содержит ссылку* на некоторое значение.

Вероятно, это различие ни на что не повлияет в ваших первых программах. И все же лучше узнать о нем раньше, чем позже. В какой-то момент вы столкнетесь с неожиданным поведением переменных, и более точное понимание работы переменных поможет вам разобраться в том, что же происходит в вашем коде.

ПРИМЕЧАНИЕ Как лучше всего освоить новые концепции программирования? Попробуйте использовать их в своей программе. Если в ходе работы над упражнением вы зайдете в тупик, попробуйте на какое-то время заняться чем-нибудь другим. Если это не поможет, перечитайте соответствующую часть этой главы. Если и это не помогло, обращайтесь к рекомендациям из приложения В.

УПРАЖНЕНИЯ

Напишите отдельную программу для выполнения каждого из следующих упражнений. Сохраните каждую программу в файле, имя которого подчиняется стандартным правилам Python по использованию строчных букв и символов подчеркивания — например, `simple_message.py` и `simple_messages.py`.

2.1. Простое сообщение: сохраните текстовое сообщение в переменной и выведите его на экран.

2.2. Простые сообщения: сохраните сообщение в переменной и выведите это сообщение. Затем замените значение переменной другим сообщением и выведите новое сообщение.

Строки

Так как многие программы определяют и собирают некие данные, а затем делают с ними что-то полезное, желательно выделить основные разновидности данных. Начнем со строковых данных. На первый взгляд строки достаточно просты, но с ними можно работать многими разными способами.

Строка представляет собой простую последовательность символов. Любая последовательность символов, заключенная в кавычки, в Python считается строкой; при этом строки могут быть заключены как в одиночные, так и в двойные кавычки:

```
"This is a string."
'This is also a string.'
```

Это правило позволяет использовать внутренние кавычки и апострофы в строках:

```
'I told my friend, "Python is my favorite language!'"
"The language 'Python' is named after Monty Python, not the snake."
"One of Python's strengths is its diverse and supportive community."
```

Рассмотрим некоторые типичные операции со строками.

Изменение регистра символов в строках

Одна из простейших операций, выполняемых со строками, — изменение регистра символов. Взгляните на следующий фрагмент кода и попробуйте определить, что в нем происходит:

```
name.py
name = "ada lovelace"
print(name.title())
```

Сохраните файл с именем `name.py` и запустите его. Вывод программы должен выглядеть так:

```
Ada Lovelace
```

В этом примере в переменной `name` сохраняется строка, состоящая из букв нижнего регистра `"ada lovelace"`. За именем переменной в команде `print()` следует вызов метода `title()`. Метод представляет собой действие, которое Python выполняет с данными. Точка `(.)` после `name` в конструкции `name.title()` приказывает Python применить метод `title()` к переменной `name`. За именем метода всегда следует пара круглых скобок, потому что методам для выполнения их работы часто требуется дополнительная информация. Эта информация указывается в скобках. Функции `title()` дополнительная информация не нужна, поэтому в круглых скобках ничего нет.

Метод `title()` преобразует первый символ каждого слова в строке к верхнему регистру, тогда как все остальные символы выводятся в нижнем регистре. Например, данная возможность может быть полезна, если в вашей программе входные значения `Ada`, `ADA` и `ada` должны рассматриваться как одно и то же имя и все они должны отображаться в виде `Ada`.

Для работы с регистром также существуют другие полезные методы. Например, все символы строки можно преобразовать к верхнему или нижнему регистру:

```
name = "Ada Lovelace"
print(name.upper())
print(name.lower())
```

Программа выводит следующий результат:

```
ADA LOVELACE
ada lovelace
```

Метод `lower()` особенно полезен для хранения данных. Нередко программист не может рассчитывать на то, что пользователи введут все данные с точным соблюдением регистра, поэтому строки перед сохранением преобразуются к нижнему регистру. Затем, когда потребуется вывести информацию, используется регистр, наиболее подходящий для каждой строки.

Использование переменных в строках

В некоторых ситуациях требуется использовать значения переменных внутри строки. Представьте, что имя и фамилия хранятся в разных переменных и вы хотите объединить их для вывода полного имени:

full_name.py

```
first_name = "ada"
last_name = "lovelace"
❶ full_name = f"{first_name} {last_name}"
print(full_name)
```

Чтобы вставить значение переменной в строку, поставьте букву `f` непосредственно перед открывающей кавычкой ❶. Заключите имя (или имена) переменных, кото-

рые должны использоваться внутри строки, в фигурные скобки. Python заменит каждую переменную ее значением при выводе строки.

Такие строки называются *f-строками*. Буква *f* происходит от слова «format», потому что Python форматирует строку, заменяя имена переменных в фигурных скобках их значениями. Приведенный код выводит следующий результат:

```
ada lovelace
```

С *f*-строками можно сделать много интересного. Например, с их помощью можно строить сложные сообщения с информацией, хранящейся в переменных. Рассмотрим пример:

```
first_name = "ada"
last_name = "lovelace"
full_name = f"{first_name} {last_name}"
❶ print(f"Hello, {full_name.title()}!")
```

Полное имя используется в точке ❶ для вывода приветственного сообщения, а метод `title()` обеспечивает правильное форматирование имени. Этот фрагмент возвращает простое, хорошо отформатированное сообщение:

```
Hello, Ada Lovelace!
```

F-строками также можно воспользоваться для построения сообщения, которое затем сохраняется в переменной:

```
first_name = "ada"
last_name = "lovelace"
full_name = f"{first_name} {last_name}"
❶ message = f"Hello, {full_name.title()}!"
❷ print(message)
```

Этот код также выводит сообщение «Hello, Ada Lovelace!», но сохранение текста сообщения в переменной ❶ существенно упрощает завершающую команду печати ❷.

ПРИМЕЧАНИЕ *F*-строки впервые появились в Python 3.6. Если вы используете Python 3.5 или более раннюю версию, используйте метод `format()` вместо синтаксиса *f*-строк. Чтобы использовать `format()`, перечислите переменные, которые должны использоваться, в строке, в круглых скобках после `format`. Каждая переменная обозначается парой фигурных скобок; эти позиции будут заполняться значениями, перечисленными в круглых скобках в указанном порядке:

```
full_name = "{} {}".format(first_name, last_name)
```

Табуляции и разрывы строк

В программировании термином *пропуск* (whitespace) называются такие непечатаемые символы, как пробелы, табуляции и символы конца строки. Пропуски структурируют текст, чтобы пользователю было удобнее читать его.

Для включения в текст позиции табуляции используется комбинация символов `\t`, как в точке ❶:

```
>>> print("Python")
Python
❶ >>> print("\tPython")
Python
```

Разрывы строк добавляются с помощью комбинации символов `\n`:

```
>>> print("Languages:\nPython\nC\nJavaScript")
Languages:
Python
C
JavaScript
```

Табуляции и разрывы строк могут сочетаться в тексте. Скажем, последовательность `"\n\t"` приказывает Python начать текст с новой строки, в начале которой располагается табуляция. Следующий пример демонстрирует вывод одного сообщения с разбиением на четыре строки:

```
>>> print("Languages:\n\tPython\n\tC\n\tJavaScript")
Languages:
Python
C
JavaScript
```

Разрывы строк и табуляции часто встречаются в двух следующих главах, когда наши программы начнут выводить относительно длинный текст.

Удаление пропусков

Лишние пропуски могут вызвать путаницу в программах. Для программиста строки `'python'` и `'python '` внешне неотличимы, но для программы это совершенно разные строки. Python видит лишний пробел в `'python '` и считает, что он действительно важен — до тех пор, пока вы не сообщите о противоположном.

Обращайте внимание на пропуски, потому что в программах часто приходится сравнивать строки, чтобы проверить на совпадение их содержимое. Типичный пример — проверка имен пользователей при входе на сайт. Лишние пропуски могут создавать путаницу и в более простых ситуациях. К счастью, Python позволяет легко удалить лишние пропуски из данных, введенных пользователем.

Python может искать лишние пропуски у левого и правого края строки. Чтобы убедиться в том, что у правого края (в конце) строки нет пропусков, вызовите метод `rstrip()`.

```
❶ >>> favorite_language = 'python '
❷ >>> favorite_language
'python '
❸ >>> favorite_language.rstrip()
'python'
❹ >>> favorite_language
'python '
```

Значение, хранящееся в переменной `favorite_language` в точке ❶, содержит лишние пропуски в конце строки. Когда вы приказываете Python вывести это значение в терминальном сеансе, вы видите пробел в конце значения ❷. Когда метод `rstrip()` работает с переменной `favorite_language` в точке ❸, этот лишний пробел удаляется. Впрочем, удаление лишь временное — если снова запросить значение `favorite_language`, мы видим, что строка не отличается от исходной, включая лишний пропуск ❹.

Чтобы навсегда исключить пропуск из строки, следует записать усеченное значение обратно в переменную:

```
>>> favorite_language = 'python '
❶ >>> favorite_language = favorite_language.rstrip()
>>> favorite_language
'python'
```

Сначала пропуски удаляются в конце строки, а потом значение записывается в исходную переменную ❶. Операция изменения значения переменной с последующим его сохранением в исходной переменной часто выполняется в программировании. Так, значение переменной может изменяться в ходе выполнения программы или в ответ на действия пользователя.

Пропуски также можно удалить у левого края (в начале) строки при помощи метода `lstrip()`, а метод `strip()` удаляет пропуски с обоих концов:

```
❶ >>> favorite_language = ' python '
❷ >>> favorite_language.rstrip()
' python'
❸ >>> favorite_language.lstrip()
'python '
❹ >>> favorite_language.strip()
'python'
```

В этом примере исходное значение содержит пропуски в начале и в конце ❶. Затем пропуски удаляются у правого края ❷, у левого края ❸ и с обоих концов строки ❹. Поэкспериментируйте с функциями удаления пропусков, это поможет вам освоиться с работой со строками. На практике эти функции чаще всего применяются для «очистки» пользовательского ввода перед его сохранением в программе.

Предотвращение синтаксических ошибок в строках

Синтаксические ошибки встречаются в программах относительно регулярно. *Синтаксическая ошибка* происходит тогда, когда Python не распознает часть вашей программы как действительный код. Например, если заключить апостроф в одиночные кавычки, произойдет ошибка. Это происходит из-за того, что Python интерпретирует все символы от первой одиночной кавычки до апострофа как строку. После этого Python пытается интерпретировать остаток текста строки как код Python, что порождает ошибки.

Разберемся, как же правильно использовать одиночные или двойные кавычки. Сохраните следующую программу в файле `apostrophe.py` и запустите ее:

`apostrophe.py`

```
message = "One of Python's strengths is its diverse community."  
print(message)
```

Апостроф находится в строке, заключенной в двойные кавычки, так что у интерпретатора Python не возникает проблем с правильной интерпретацией следующей строки:

```
One of Python's strengths is its diverse community.
```

Однако при использовании одиночных кавычек Python не сможет определить, где должна заканчиваться строка:

```
message = 'One of Python's strengths is its diverse community.'  
print(message)
```

Программа выводит следующий результат:

```
File "apostrophe.py", line 1  
    message = 'One of Python's strengths is its diverse community.'  
                ^❶
```

SyntaxError: invalid syntax

Из выходных данных видно, что ошибка происходит в позиции **❶** сразу же после второй одиночной кавычки. Эта синтаксическая ошибка указывает, что интерпретатор не распознает какую-то конструкцию как действительный код Python. Ошибки могут возникать по разным причинам; я буду выделять наиболее распространенные источники по мере того, как они будут встречаться нам. Синтаксические ошибки будут часто встречаться вам в то время, пока вы учитесь писать правильный код Python. Кроме того, ошибки этой категории также являются наиболее расплывчатыми и неконкретными, поэтому их особенно трудно находить и исправлять. Если вы зайдете в тупик из-за особенно коварной ошибки, обращайтесь к рекомендациям в приложении В.

ПРИМЕЧАНИЕ Функция цветового выделения синтаксиса ускоряет выявление некоторых синтаксических ошибок прямо во время написания программы. Если вы увидите, что код Python выделяется как обычный текст (или обычный текст выделяется как код Python), скорее всего, в вашем файле где-то пропущена кавычка.

УПРАЖНЕНИЯ

Сохраните код каждого из следующих упражнений в отдельном файле с именем вида `name_cases.py`. Если у вас возникнут проблемы, сделайте перерыв или обратитесь к рекомендациям в приложении В.

2.3. Личное сообщение: сохраните имя пользователя в переменной и выведите сообщение, предназначенное для конкретного человека. Сообщение должно быть простым — например, «Hello Eric, would you like to learn some Python today?».

3

Списки

В этой и следующей главе вы узнаете, что собой представляют списки и как начать работать с элементами списка. Списки позволяют хранить в одном месте взаимосвязанные данные, сколько бы их ни было — несколько элементов или несколько миллионов элементов. Работа со списками принадлежит к числу самых выдающихся возможностей Python, доступных для начинающего программиста. Операции со списками связывают воедино многие важные концепции в программировании.

Что такое список?

Список представляет собой набор элементов, следующих в определенном порядке. Вы можете создать список для хранения букв алфавита, цифр от 0 до 9 или имен всех членов вашей семьи. В список можно поместить любую информацию, причем данные в списке даже не обязаны быть как-то связаны друг с другом. Так как список обычно содержит более одного элемента, рекомендуется присваивать спискам имена во множественном числе: **letters**, **digits**, **names** и т. д.

В языке Python список обозначается квадратными скобками (`[]`), а отдельные элементы списка разделяются запятыми. Простой пример списка с названиями моделей велосипедов:

bicycles.py

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles)
```

Если вы прикажете Python вывести список, то на экране появится перечисление элементов списка в квадратных скобках:

```
['trek', 'cannondale', 'redline', 'specialized']
```

Конечно, вашим пользователям такое представление не подойдет; разберемся, как обратиться отдельным элементам в списке.

Обращение к элементам списка

Списки представляют собой упорядоченные наборы данных, поэтому для обращения к любому элементу списка следует сообщить Python позицию (*индекс*) нужного элемента. Чтобы обратиться к элементу в списке, укажите имя списка, за которым следует индекс элемента в квадратных скобках.

Например, название первого велосипеда в списке `bicycles` выводится следующим образом:

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']
❶ print(bicycles[0])
```

Синтаксис обращения к элементу показан в точке ❶. Когда мы запрашиваем один элемент из списка, Python возвращает только этот элемент без квадратных скобок или кавычек:

```
trek
```

Именно такой результат должны увидеть пользователи — чистый, аккуратно отформатированный вывод.

Также можно использовать строковые методы из главы 2 с любым элементом списка. Например, элемент `'trek'` можно более аккуратно отформатировать при помощи метода `title()`:

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']
print(bicycles[0].title())
```

Этот пример выдает такой же результат, как и предыдущий, только название `'Trek'` выводится с прописной буквы.

Индексы начинаются с 0, а не с 1

Python считает, что первый элемент списка находится в позиции 0, а не в позиции 1. Этот принцип встречается в большинстве языков программирования и объясняется особенностями низкоуровневой реализации операций со списками. Если вы получаете неожиданные результаты, определите, не допустили ли вы простую ошибку «смещения на 1».

Второму элементу списка соответствует индекс 1. В этой простой схеме индекс любого элемента вычисляется уменьшением на 1 его позиции в списке. Например, чтобы обратиться к четвертому элементу списка, следует запросить элемент с индексом 3.

В следующем примере выводятся названия велосипедов с индексами 1 и 3:

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']
print(bicycles[1])
print(bicycles[3])
```

При этом выводятся второй и четвертый элементы списка:

```
cannondale  
specialized
```

В Python также существует специальный синтаксис для обращения к последнему элементу списка. Если запросить элемент с индексом `-1`, Python всегда возвращает последний элемент в списке:

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles[-1])
```

Фрагмент вернет значение `'specialized'`. Этот синтаксис весьма полезен, потому что при работе со списками часто требуется обратиться к последним элементам, не зная точного количества элементов в списке. Синтаксис также распространяется на другие отрицательные значения индексов. Индекс `-2` возвращает второй элемент от конца списка, индекс `-3` — третий элемент от конца, и т. д.

Использование отдельных элементов из списка

Отдельные значения из списка используются так же, как и любые другие переменные. Например, вы можете воспользоваться f-строками для построения сообщения, содержащего значение из списка.

Попробуем извлечь название первого велосипеда из списка и составить сообщение, включающее это значение.

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
❶ message = f"My first bicycle was a {bicycles[0].title()}."  
  
print(message)
```

В точке ❶ программа строит сообщение, содержащее значение из `bicycles[0]`, и сохраняет его в переменной `message`. Так создается простое предложение с упоминанием первого велосипеда из списка:

```
My first bicycle was a Trek.
```

УПРАЖНЕНИЯ

Попробуйте написать несколько коротких программ, чтобы получить предварительное представление о списках Python. Возможно, для упражнений каждой главы стоит создать отдельную папку, чтобы избежать неразберихи.

3.1. Имена: сохраните имена нескольких своих друзей в списке с именем `names`. Выведите имя каждого друга, обратившись к каждому элементу списка (по одному за раз).

3.2. Сообщения: начните со списка, использованного в упражнении 3.1, но вместо вывода имени каждого человека выведите сообщение. Основной текст всех сообщений должен быть одинаковым, но каждое сообщение должно включать имя адресата.

3.3. Собственный список: выберите свой любимый вид транспорта (например, мотоциклы или машины) и создайте список с примерами. Используйте свой список для вывода утверждений об элементах типа: «Я хотел бы купить мотоцикл Honda».

Изменение, добавление и удаление элементов

Как правило, вы будете создавать *динамические* списки; это означает, что во время выполнения программы в созданном вами списке будут добавляться и удаляться элементы. Например, вы можете создать игру, в которой игрок должен стрелять по кораблям космических захватчиков. Исходный набор кораблей сохраняется в списке; каждый раз, когда вы сбиваете корабль, он удаляется из списка. Каждый раз, когда на экране появляется новый враг, он включается в список. Длина списка кораблей будет уменьшаться и увеличиваться по ходу игры.

Изменение элементов в списке

Синтаксис изменения элемента напоминает синтаксис обращения к элементу списка. Чтобы изменить элемент, укажите имя списка и индекс изменяемого элемента в квадратных скобках; далее задайте новое значение, которое должно быть присвоено элементу.

Допустим, имеется список мотоциклов и первым элементом списка хранится строка 'honda'. Как изменить значение первого элемента?

motorcycles.py

```
❶ motorcycles = ['honda', 'yamaha', 'suzuki']
   print(motorcycles)

❷ motorcycles[0] = 'ducati'
   print(motorcycles)
```

В точке ❶ определяется исходный список, в котором первый элемент содержит строку 'honda'. В точке ❷ значение первого элемента заменяется строкой 'ducati'. Из вывода видно, что первый элемент действительно изменился, тогда как остальные элементы списка сохранили прежние значения:

```
['honda', 'yamaha', 'suzuki']
['ducati', 'yamaha', 'suzuki']
```

Изменить можно значение любого элемента в списке, не только первого.

Добавление элементов в список

Новые элементы могут добавляться в списки по разным причинам: например, для появления на экране новых космических кораблей, включения новых данных в визуализацию или добавления новых зарегистрированных пользователей на по-

строенный вами сайт. Python предоставляет несколько способов добавления новых данных в существующие списки.

Присоединение элементов в конец списка

Простейший способ добавления новых элементов в список — *присоединение* элемента в конец списка. Используя список из предыдущего примера, добавим новый элемент `'ducati'` в конец списка:

```
motorcycles = ['honda', 'yamaha', 'suzuki']
print(motorcycles)
❶ motorcycles.append('ducati')
print(motorcycles)
```

Метод `append()` в точке ❶ присоединяет строку `'ducati'` в конец списка, другие элементы в списке при этом остаются неизменными:

```
['honda', 'yamaha', 'suzuki']
['honda', 'yamaha', 'suzuki', 'ducati']
```

Метод `append()` упрощает динамическое построение списков. Например, вы можете начать с пустого списка и добавлять в него элементы серией команд `append()`. В следующем примере в пустой список добавляются элементы `'honda'`, `'yamaha'` и `'suzuki'`:

```
motorcycles = []
motorcycles.append('honda')
motorcycles.append('yamaha')
motorcycles.append('suzuki')
print(motorcycles)
```

Полученный список выглядит точно так же, как и списки из предыдущих примеров:

```
['honda', 'yamaha', 'suzuki']
```

Такой способ построения списков встречается очень часто, потому что данные, которые пользователь захочет сохранить в программе, часто становятся известными только после запуска программы. Чтобы пользователь мог управлять содержимым списка, начните с определения пустого списка, а затем присоединяйте к нему каждое новое значение.

Вставка элементов в список

Метод `insert()` позволяет добавить новый элемент в произвольную позицию списка. Для этого следует указать индекс и значение нового элемента.

```
motorcycles = ['honda', 'yamaha', 'suzuki']
❶ motorcycles.insert(0, 'ducati')
print(motorcycles)
```

В этом примере в точке ❶ значение `'ducati'` вставляется в начало списка. Метод `insert()` выделяет свободное место в позиции 0 и сохраняет в нем значение `'ducati'`. Все остальные значения списка при этом сдвигаются на одну позицию вправо:

```
['ducati', 'honda', 'yamaha', 'suzuki']
```

Удаление элементов из списка

Нередко возникает необходимость в удалении одного или нескольких элементов из списка. Например, когда игрок сбивает корабль пришельца, этот корабль стоит удалить из списка активных врагов. Или когда пользователь решает удалить свою учетную запись в созданном вами веб-приложении, этот пользователь должен быть удален из списка активных пользователей. Элементы удаляются из списка по позиции или по значению.

Удаление элемента с использованием команды `del`

Если вам известна позиция элемента, который должен быть удален из списка, воспользуйтесь командой `del`.

```
motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles)
```

```
❶ del motorcycles[0]  
print(motorcycles)
```

В точке ❶ вызов `del` удаляет первый элемент, `'honda'`, из списка `motorcycles`:

```
['honda', 'yamaha', 'suzuki']  
['yamaha', 'suzuki']
```

Команда `del` позволяет удалить элемент из любой позиции списка, если вам известен его индекс. Например, вот как из списка удаляется второй элемент, `'yamaha'`:

```
motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles)
```

```
del motorcycles[1]  
print(motorcycles)
```

Второй элемент исчез из списка:

```
['honda', 'yamaha', 'suzuki']  
['honda', 'suzuki']
```

В обоих примерах значение, удаленное из списка после использования команды `del`, становится недоступным.

Удаление элемента с использованием метода pop()

Иногда значение, удаляемое из списка, должно как-то использоваться. Допустим, вы хотите получить координаты x и y только что сбитого корабля пришельцев, чтобы изобразить взрыв в этой позиции. В веб-приложении пользователь, удаленный из списка активных участников, может быть добавлен в список неактивных, и т. д.

Метод `pop()` удаляет последний элемент из списка, но позволяет работать с ним после удаления. Удалим мотоцикл из списка:

```
❶ motorcycles = ['honda', 'yamaha', 'suzuki']
   print(motorcycles)

❷ popped_motorcycle = motorcycles.pop()
❸ print(motorcycles)
❹ print(popped_motorcycle)
```

Сначала в точке ❶ определяется и выводится содержимое списка `motorcycles`. В точке ❷ значение извлекается из списка и сохраняется в переменной с именем `popped_motorcycle`. Вывод измененного списка в точке ❸ показывает, что значение было удалено из списка. Затем мы выводим извлеченное значение в точке ❹, демонстрируя, что удаленное из списка значение остается доступным в программе.

Из вывода видно, что значение `'suzuki'`, удаленное в конце списка, теперь хранится в переменной `popped_motorcycle`:

```
['honda', 'yamaha', 'suzuki']
['honda', 'yamaha']
suzuki
```

Для чего может понадобиться метод `pop()`? Представьте, что мотоциклы в списке хранятся в хронологическом порядке в соответствии с датой их покупки. В таком случае команда `pop()` может использоваться для вывода сообщения о последнем купленном мотоцикле:

```
motorcycles = ['honda', 'yamaha', 'suzuki']

last_owned = motorcycles.pop()
print(f"The last motorcycle I owned was a {last_owned.title()}.")
```

Программа выводит простое сообщение:

```
The last motorcycle I owned was a Suzuki.
```

Извлечение элементов из произвольной позиции списка

Вызов `pop()` может использоваться для удаления элемента в произвольной позиции списка; для этого следует указать индекс удаляемого элемента в круглых скобках.

```
motorcycles = ['honda', 'yamaha', 'suzuki']

❶ first_owned = motorcycles.pop(0)
❷ print(f"The first motorcycle I owned was a {first_owned.title()}.")
```

Сначала первый элемент извлекается из списка в точке ❶, а затем в точке ❷ выводится сообщение об этом мотоцикле. Программа выводит простое сообщение о первом мотоцикле:

```
The first motorcycle I owned was a Honda.
```

Помните, что после каждого вызова `pop()` элемент, с которым вы работаете, уже не находится в списке.

Если вы не уверены в том, какой из двух способов выбрать, команду `del` или метод `pop()`, — простое правило поможет вам определиться: если вы собираетесь просто удалить элемент из списка, никак не используя его, выбирайте команду `del`; если же вы намерены использовать элемент после удаления из списка, выбирайте метод `pop()`.

Удаление элементов по значению

Иногда позиция удаляемого элемента неизвестна. Если вы знаете только значение элемента, используйте метод `remove()`.

Допустим, из списка нужно удалить значение `'ducati'`:

```
motorcycles = ['honda', 'yamaha', 'suzuki', 'ducati']
print(motorcycles)
```

```
❶ motorcycles.remove('ducati')
   print(motorcycles)
```

Код в точке ❶ приказывает Python определить, в какой позиции списка находится значение `'ducati'`, и удалить этот элемент:

```
['honda', 'yamaha', 'suzuki', 'ducati']
['honda', 'yamaha', 'suzuki']
```

Метод `remove()` также может использоваться для работы со значением, которое удаляется из списка. Следующая программа удаляет значение `'ducati'` и выводит причину удаления:

```
❶ motorcycles = ['honda', 'yamaha', 'suzuki', 'ducati']
   print(motorcycles)

❷ too_expensive = 'ducati'
❸ motorcycles.remove(too_expensive)
   print(motorcycles)
❹ print(f"\nA {too_expensive.title()} is too expensive for me.")
```

После определения списка в точке ❶ значение `'ducati'` сохраняется в переменной с именем `too_expensive` в точке ❷. Затем эта переменная сообщает Python, какое значение должно быть удалено из списка ❸. В точке ❹ значение `'ducati'` было удалено из списка, но продолжает храниться в переменной `too_expensive`,

что позволяет вывести сообщение с причиной удаления 'ducati' из списка мотоциклов:

```
['honda', 'yamaha', 'suzuki', 'ducati']  
['honda', 'yamaha', 'suzuki']
```

A Ducati is too expensive for me.

ПРИМЕЧАНИЕ Метод `remove()` удаляет только первое вхождение заданного значения. Если существует вероятность того, что значение встречается в списке более одного раза, используйте цикл для определения того, были ли удалены все вхождения данного значения. О том, как это делать, рассказано в главе 7.

УПРАЖНЕНИЯ

Следующие упражнения немного сложнее упражнений из главы 2, но они предоставляют возможность попрактиковаться в выполнении всех описанных операций со списками.

3.4. Список гостей: если бы вы могли пригласить кого угодно (из живых или умерших) на обед, то кого бы вы пригласили? Создайте список, включающий минимум трех людей, которых вам хотелось бы пригласить на обед. Затем используйте этот список для вывода пригласительного сообщения каждому участнику.

3.5. Изменение списка гостей: вы только что узнали, что один из гостей прийти не сможет, поэтому вам придется разослать новые приглашения. Отсутствующего гостя нужно заменить кем-то другим.

- Начните с программы из упражнения 3.4. Добавьте в конец программы команду `print` для вывода имени гостя, который прийти не сможет.
- Измените список и замените имя гостя, который прийти не сможет, именем нового приглашенного.
- Выведите новый набор сообщений с приглашениями — по одному для каждого участника, входящего в список.

3.6. Больше гостей: вы решили купить обеденный стол большего размера. Дополнительные места позволяют пригласить на обед еще трех гостей.

- Начните с программы из упражнения 3.4 или 3.5. Добавьте в конец программы команду `print`, которая выводит сообщение о расширении списка гостей.
- Добавьте вызов `insert()` для добавления одного гостя в начало списка.
- Добавьте вызов `insert()` для добавления одного гостя в середину списка.
- Добавьте вызов `append()` для добавления одного гостя в конец списка.
- Выведите новый набор сообщений с приглашениями — по одному для каждого участника, входящего в список.

3.7. Сокращение списка гостей: только что выяснилось, что новый обеденный стол привезти вовремя не успеют и места хватит только для двух гостей.

- Начните с программы из упражнения 3.6. Добавьте команду для вывода сообщения о том, что на обед приглашаются всего два гостя.
- Используйте метод `pop()` для последовательного удаления гостей из списка до тех пор, пока в списке не останутся только два человека. Каждый раз, когда из списка удаляется очередное имя, выведите для этого человека сообщение о том, что вы сожалеете об отмене приглашения.

- Выведите сообщение для каждого из двух человек, остающихся в списке. Сообщение должно подтверждать, что более раннее приглашение остается в силе.
- Используйте команду `del` для удаления двух последних имен, чтобы список остался пустым. Выведите список, чтобы убедиться в том, что в конце работы программы список действительно не содержит ни одного элемента.

Упорядочение списка

Нередко список создается в непредсказуемом порядке, потому что порядок получения данных от пользователя не всегда находится под вашим контролем. И хотя во многих случаях такое положение дел неизбежно, часто требуется вывести информацию в определенном порядке. В одних случаях требуется сохранить исходный порядок элементов в списке, в других исходный порядок должен быть изменен. Python предоставляет в распоряжение программиста несколько разных способов упорядочения списка в зависимости от ситуации.

Постоянная сортировка списка методом `sort()`

Метод `sort()` позволяет относительно легко отсортировать список. Предположим, имеется список машин и вы хотите переупорядочить эти элементы по алфавиту. Чтобы упростить задачу, предположим, что все значения в списке состоят из символов нижнего регистра.

cars.py

```
cars = ['bmw', 'audi', 'toyota', 'subaru']
❶ cars.sort()
print(cars)
```

Метод `sort()` в точке ❶ осуществляет постоянное изменение порядка элементов в списке. Названия машин располагаются в алфавитном порядке, и вернуться к исходному порядку уже не удастся:

```
['audi', 'bmw', 'subaru', 'toyota']
```

Список также можно отсортировать в обратном алфавитном порядке; для этого методу `sort()` следует передать аргумент `reverse=True`. В следующем примере список сортируется в порядке, обратном алфавитному:

```
cars = ['bmw', 'audi', 'toyota', 'subaru']
cars.sort(reverse=True)
print(cars)
```

И снова порядок элементов изменяется на постоянной основе:

```
['toyota', 'subaru', 'bmw', 'audi']
```

Временная сортировка списка функцией `sorted()`

Чтобы сохранить исходный порядок элементов списка, но временно представить их в отсортированном порядке, можно воспользоваться функцией `sorted()`. Функция `sorted()` позволяет представить список в определенном порядке, но не изменяет фактический порядок элементов в списке.

Попробуем применить эту функцию к списку машин.

```
cars = ['bmw', 'audi', 'toyota', 'subaru']

❶ print("Here is the original list:")
   print(cars)

❷ print("\nHere is the sorted list:")
   print(sorted(cars))

❸ print("\nHere is the original list again:")
   print(cars)
```

Сначала список выводится в исходном порядке ❶, а затем в алфавитном порядке ❷. После того как список будет выведен в новом порядке, в точке ❸ мы убеждаемся в том, что список все еще хранится в исходном порядке.

```
Here is the original list:
['bmw', 'audi', 'toyota', 'subaru']

Here is the sorted list:
['audi', 'bmw', 'subaru', 'toyota']

❹ Here is the original list again:
['bmw', 'audi', 'toyota', 'subaru']
```

Обратите внимание: после вызова функции `sorted()` список продолжает храниться в исходном порядке ❹. Функции `sorted()` также можно передать аргумент `reverse=True`, чтобы список был представлен в порядке, обратном алфавитному.

ПРИМЕЧАНИЕ Если не все значения записаны в нижнем регистре, алфавитная сортировка списка немного усложняется. При определении порядка сортировки появляются разные способы интерпретации прописных букв, и точное определение порядка уже не столь тривиально (во всяком случае, чтобы отвлекаться на него сейчас). Впрочем, большинство способов сортировки напрямую следует из того, о чем вы узнали в этом разделе.

Вывод списка в обратном порядке

Чтобы переставить элементы списка в обратном порядке, используйте метод `reverse()`. Скажем, если список машин первоначально хранился в хронологическом порядке даты приобретения, элементы можно легко переупорядочить в обратном хронологическом порядке:

Эрик Мэттиз

**Изучаем Python: программирование игр,
визуализация данных, веб-приложения**

3-е издание

Перевел с английского *Е. Матвеев*

Заведующая редакцией	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>К. Тульцева</i>
Художественный редактор	<i>В. Мостипан</i>
Литературный редактор	<i>М. Рогожин</i>
Корректоры	<i>С. Беляева, Н. Викторова</i>
Верстка	<i>Л. Егорова</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».

Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 07.2020. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12.000 — Книги печатные
профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 08.07.20. Формат 70×100/16. Бумага офсетная. Усл. п. л. 41,280. Доп. тираж. Заказ 0000.